

NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES**FAST SCHOOL OF COMPUTING**

Department of Computer Science

**Data Structures****Lab Manual**

Lab Instructor	Mr. Jawad Khan
Section	BCS-3H
Semester	Fall 2026

Lab 04: Circular Linked Lists and Iterator-Based List Algorithms

1. Learning Objectives

After completing this lab, students will be able to:

1. Understand how a circular linked list differs from a linear one, and why traversal and midpoint-finding logic must change as a result.
2. Split a circular singly linked list into two balanced circular halves.
3. Understand the iterator abstraction and why it lets algorithms operate on a list without accessing its internal node structure.
4. Extend a doubly linked list ADT with iterator support (begin, end, and the core iterator operations).
5. Design global, iterator-based functions that compute the union, difference, and intersection of two doubly linked lists.
6. Compare two doubly linked lists for equality using iterators.
7. Remove duplicate values from an unsorted doubly linked list while preserving order.
8. Understand how a circular doubly linked list enables rotation without relinking any pointers.

2. Lab Instructions

- Create a separate .cpp file for each task.
- Use meaningful variable, function, and class names.
- Properly indent and format your code.
- Use meaningful prompts and output labels.
- Add comments where the logic is not immediately obvious.
- Do not use global variables unless explicitly required.
- Do not use STL containers such as vector, list, set, or map unless explicitly permitted.
- Dynamically allocated memory must be properly released.
- Make sure there are no memory leaks or dangling pointers.
- Test your program with normal, boundary, and invalid inputs.
- For tasks containing a time-complexity constraint, the required complexity must be followed.

3. Theory and Concept Review

3.1 Recap: The Doubly Linked List ADT

In the previous lab, you implemented a `DoublyLinkedList<T>` class template with a nested `Node` class holding data, prev, and next, along with `insertAtHead`, `insertAtTail`, `deleteAtHead`, `deleteAtTail`, `display`, `displayReverse`, `isPalindrome`, and `deleteNode`. This lab builds directly on that class.

3.2 What Is a Circular Linked List?

In a linear singly linked list, the last node's next pointer is `nullptr`, marking the end of the list. In a circular linked list, the last node's next pointer instead points back to the head, so the nodes form a ring with no true end. Traversal has to account for this: instead of stopping when a pointer becomes `nullptr`, code must stop when it arrives back at a node it has already visited, typically the head.

This has a practical consequence for any technique that relies on stopping at `nullptr`. The two-pointer technique used in a previous lab to find the middle of a list stopped once the faster pointer reached `nullptr`. In a circular list, the faster pointer would instead loop around the ring forever, so the stopping condition has to be rewritten in terms of returning to the head rather than reaching `nullptr`.

3.3 What Is an Iterator?

An iterator is a small object that represents a position within a container, such as a linked list, and knows how to move to the next or previous position and retrieve the value stored there. Code that uses an iterator never touches the container's internal node structure directly; it only calls the iterator's own operations.

A doubly linked list's iterator is typically written as a small nested class wrapping a single `Node` pointer:

```
class Iterator {
public:
    T& operator*();           // access the value at this position
    Iterator& operator++();    // move forward one position
    Iterator& operator--();    // move backward one position
    bool operator==(const Iterator& other) const;
    bool operator!=(const Iterator& other) const;
    Node* node() const;       // expose the underlying node handle

    // ... you will design the body of each of these in Task 2
};
```

A list exposes two functions that produce iterators: `begin()`, which refers to the first node, and `end()`, which represents one position past the last node rather than any real node. Code can then walk the whole list with a loop such as `for (auto it = list.begin(); it != list.end(); ++it)`, without ever mentioning `Node` directly.

3.4 Why Write These as Global Functions?

The union, difference, intersection, equality, and duplicate-removal operations in this lab are written as free functions parameterized by iterators, rather than as member functions of `DoublyLinkedList`. This mirrors how the C++ Standard Library's own algorithms work, such as merging or comparing two ranges: an algorithm written against iterators does not need to know anything about the specific container it is operating on, only how to move an iterator forward and read the value at its current position. Writing an

algorithm once against the iterator interface, rather than once per container type, is the central idea behind this style of generic programming.

3.5 Circular Doubly Linked Lists

A circular doubly linked list combines both ideas: every node has a prev and next pointer as in a regular doubly linked list, but the last node's next points back to the head, and the head's prev points back to the last node, so the ring is fully connected in both directions.

This has a useful consequence for operations that would otherwise require relinking pointers. Because every node is already connected to its neighbors all the way around the ring, some operations that need to change the shape of a linear list can instead be done by simply changing which node is treated as the head, without moving or relinking a single node.

4. Lab Tasks

Task 1: Circular Singly Linked List, Split into Two Halves

Implement a class template named `CircularLinkedList` that manages a circular singly linked list of any data type, with the node implemented as a nested `Node` class. This is your first task involving a circular list, so the required insertion and display functions are listed alongside the split operation below. Do not use `vector`, `list`, or any other STL container.

Required Node Structure

```
class Node {
public:
    T data;
    Node* next;
};
```

Required Functions

Each row states only what the function must do; designing how it does it is part of the task.

<code>void insertAtEnd(T value);</code>	Design a function that inserts a new node containing value at the end of the list, correctly maintaining the circular link back to the head.
<code>void display();</code>	Design a function that prints every node's value exactly once, starting from the head.
<code>int size();</code>	Design a function that returns the number of nodes currently in the list.

<code>bool isEmpty();</code>	Design a function that returns whether the list currently contains any nodes.
<code>void splitInHalf(CircularLinkedList<T>& firstHalf, CircularLinkedList<T>& secondHalf);</code>	Design a function that splits this list into two circular lists of equal size, placing the results into the two lists passed in. If the original list has an odd number of nodes, the first half should contain one more node than the second half.

Complexity Requirement

- `insertAtEnd` and `display` must each run in $O(n)$.
- `splitInHalf` must locate the midpoint in a single traversal of the list; do not count the nodes in one pass and then walk to the midpoint in a second pass.

Restrictions

- Do not use any STL container.
- Each of the two resulting lists must itself be a valid, correctly closed circular list; its own last node must point back to its own head, not to the head of the original list.
- Clearly state the time complexity of `splitInHalf` in a comment.

Example

```
List (circular): 1 2 3 4 5

splitInHalf(firstHalf, secondHalf)

firstHalf: 1 2 3
secondHalf: 4 5
```

Task 2: Add Iterator Support to the Doubly Linked List

Extend your `DoublyLinkedList<T>` class from the previous lab by adding a nested `Iterator` class and the functions needed to create one, as introduced in Section 3.3. Tasks 3 through 7 all depend on this working correctly.

Required Iterator Structure

```
class Iterator {
public:
    T& operator*();
    Iterator& operator++();
    Iterator& operator--();
    bool operator==(const Iterator& other) const;
    bool operator!=(const Iterator& other) const;
    Node* node() const;
};
```

Required Functions

Each row states only what the function must do; designing how it does it is part of the task.

<code>Iterator begin();</code>	Design a function, on <code>DoublyLinkedList</code> , that returns an iterator referring to the first node of the list.
<code>Iterator end();</code>	Design a function, on <code>DoublyLinkedList</code> , that returns an iterator representing one position past the last node. This iterator must never be dereferenced; it exists only to be compared against.
<code>T& operator*();</code>	Design a function, on <code>Iterator</code> , that returns a reference to the value stored at the iterator's current position.
<code>Iterator& operator++();</code>	Design a function, on <code>Iterator</code> , that advances the iterator to the next node and returns the updated iterator.
<code>Iterator& operator--();</code>	Design a function, on <code>Iterator</code> , that moves the iterator to the previous node and returns the updated iterator.
<code>bool operator==(const Iterator& other) const;</code>	Design a function, on <code>Iterator</code> , that returns whether two iterators refer to the same position.
<code>bool operator!=(const Iterator& other) const;</code>	Design a function, on <code>Iterator</code> , that returns whether two iterators refer to different positions.
<code>Node* node() const;</code>	Design a function, on <code>Iterator</code> , that exposes the node the iterator currently refers to, so that other code, such as Task 7, can pass it to <code>deleteNode</code> .

Complexity Requirement

- Every function listed above must run in $O(1)$.

Required Behaviour

- Calling `begin()` on an empty list must produce an iterator equal to `end()`.
- A loop of the form `for (auto it = list.begin(); it != list.end(); ++it) { ... }` must visit every node in the list exactly once, in order.

Task 3: unionList

Write a global function template that takes iterators marking the start and end of two unsorted doubly linked lists, and returns a new list containing every distinct value that appears in either list.

```
template <typename T>
DoublyLinkedList<T> unionList(
    typename DoublyLinkedList<T>::Iterator begin1,
    typename DoublyLinkedList<T>::Iterator end1,
    typename DoublyLinkedList<T>::Iterator begin2,
    typename DoublyLinkedList<T>::Iterator end2
);
```

Complexity Requirement

- Time Complexity: $O(n \times m)$, where n and m are the sizes of the two input lists. Without a hash-based structure, checking whether a value has already been added requires comparing it against the values added so far.
- Space Complexity: $O(n + m)$ for the result list, in the worst case.

Restrictions

- Do not use any STL container.
- Do not modify either of the two input lists.
- The result must not contain duplicate values, even if a value appears in both input lists, or more than once within a single input list.
- Clearly state the time and space complexity of your solution in a comment.

Example

```
List A = 1 2 3
List B = 2 3 4

unionList(A.begin(), A.end(), B.begin(), B.end())

Result: 1 2 3 4
```

Task 4: differenceList

Write a global function template that takes iterators marking the start and end of two sorted doubly linked lists, and returns a new list containing every value that appears in the first list but not in the second.

```
template <typename T>
DoublyLinkedList<T> differenceList(
    typename DoublyLinkedList<T>::Iterator begin1,
    typename DoublyLinkedList<T>::Iterator end1,
    typename DoublyLinkedList<T>::Iterator begin2,
    typename DoublyLinkedList<T>::Iterator end2
);
```

Important Complexity Constraint

Because both input lists are already sorted in ascending order, you are required to compute the difference in a single simultaneous pass through both lists, advancing through each list at most once, rather than comparing every element of one list against every element of the other.

Restrictions

- Do not use any STL container.
- You may assume both input lists are already sorted in ascending order.
- Do not modify either of the two input lists.

- Clearly state the time and space complexity of your solution in a comment.

Example

```
List A = 1 2 3 4 5
List B = 2 4 6

differenceList(A.begin(), A.end(), B.begin(), B.end())

Result (A - B): 1 3 5
```

Task 5: intersectionList

Write a global function template that takes iterators marking the start and end of two unsorted doubly linked lists, and returns a new list containing only the values that appear in both lists.

```
template <typename T>
DoublyLinkedList<T> intersectionList(
    typename DoublyLinkedList<T>::Iterator begin1,
    typename DoublyLinkedList<T>::Iterator end1,
    typename DoublyLinkedList<T>::Iterator begin2,
    typename DoublyLinkedList<T>::Iterator end2
);
```

Complexity Requirement

- Time Complexity: $O(n \times m)$, for the same reason as unionList.
- Space Complexity: $O(\min(n, m))$ for the result list, in the worst case.

Restrictions

- Do not use any STL container.
- Do not modify either of the two input lists.
- The result must not contain duplicate values.
- Clearly state the time and space complexity of your solution in a comment.

Example

```
List A = 1 2 3 4
List B = 3 4 5 6

intersectionList(A.begin(), A.end(), B.begin(), B.end())

Result: 3 4
```

Task 6: equalLists

Write a global function template that compares two doubly linked lists element by element using iterators. Return true only if the lists are equal in both size and content, in the same order.

```
template <typename T>
bool equalLists(
    typename DoublyLinkedList<T>::Iterator begin1,
    typename DoublyLinkedList<T>::Iterator end1,
    typename DoublyLinkedList<T>::Iterator begin2,
    typename DoublyLinkedList<T>::Iterator end2
);
```

Complexity Requirement

- Time Complexity: $O(n)$, where n is the length of the shorter of the two lists.
- Space Complexity: $O(1)$.

Restrictions

- Do not use any STL container.
- Do not modify either of the two input lists.
- Clearly state the time and space complexity of your solution in a comment.

Required Behaviour

- Two lists of different lengths must never be reported as equal, even if every element of the shorter list matches the corresponding element of the longer one.

Example

List A = 1 2 3	List B = 1 2 3	equalLists: true
List A = 1 2 3	List B = 1 2 4	equalLists: false
List A = 1 2 3	List B = 1 2	equalLists: false

Task 7: removeDuplicates

Write a global function template that removes duplicate values from an unsorted doubly linked list, keeping only the first occurrence of each value and preserving the relative order of the remaining elements.

```
template <typename T>
void removeDuplicates(
    DoublyLinkedList<T>& list,
    typename DoublyLinkedList<T>::Iterator begin,
    typename DoublyLinkedList<T>::Iterator end
);
```

Note:

Removing an element requires more than read access to the list, so this function takes the list itself by reference in addition to the iterator range, unlike Tasks 3 through 6. Use the iterator range to walk the list

and identify duplicates, and use `node()` together with the list's existing `deleteNode` function from the previous lab to actually remove them, rather than writing new deletion logic.

Complexity Requirement

- Time Complexity: $O(n^2)$, where n is the number of nodes in the list, since each element must be checked against every element that comes after it, without a hash-based structure.
- Space Complexity: $O(1)$ extra, beyond the list itself.

Restrictions

- Do not use any STL container.
- Do not copy values into an array.
- The relative order of the remaining elements must match their original order.
- Clearly state the time and space complexity of your solution in a comment.

Example

Input: 1 2 2 3 4 3 5
Output: 1 2 3 4 5

Task 8: Circular Doubly Linked List, Rotate by k Positions

Implement a class template named `CircularDoublyLinkedList` that manages a circular doubly linked list of any data type, with the node implemented as a nested `Node` class holding data, prev, and next. As with Task 1, the required insertion and display functions are listed alongside the rotation operation below. Do not use `vector`, `list`, or any other STL container.

Required Node Structure

```
class Node {
public:
    T data;
    Node* prev;
    Node* next;
};
```

Required Functions

Each row states only what the function must do; designing how it does it is part of the task.

<code>void insertAtEnd(T value);</code>	Design a function that inserts a new node containing value at the end of the list, correctly maintaining the circular links in both directions.
<code>void display();</code>	Design a function that prints every node's value exactly once, starting from the head.

<code>int size();</code>	Design a function that returns the number of nodes currently in the list.
<code>bool isEmpty();</code>	Design a function that returns whether the list currently contains any nodes.
<code>void rotate(int k);</code>	Design a function that rotates the list by k positions, so that the node currently k positions after the head becomes the new head.

Important Complexity Constraint

As discussed in Section 3.5, a correct rotation of a circular doubly linked list should not need to relink a single pointer. You are required to implement rotate so that it only changes which node is treated as the head.

Restrictions

- Do not relink, insert, or remove any nodes as part of rotate.
- k may be larger than the number of nodes in the list, or negative; both cases must be handled correctly.
- Do not use any STL container.
- Clearly state the time complexity of rotate in a comment.

Required Behaviour

- Calling rotate on an empty list or a single-node list must have no effect.

Example

```
List (circular): 1 2 3 4 5

rotate(2)

List is now: 3 4 5 1 2
```

Submission Guidelines

Submit the following through Google Classroom:

1. Source code (.cpp file) for each completed task, i.e. 25I-XXXX_T1.cpp.
2. Screenshots showing successful execution of each task in a Word file named after your roll number.
3. Answers to the conceptual questions (If any) in the same Word document.
4. Late submissions will be penalized.

Important:

Programs that produce the correct output but violate the specified time-complexity requirement will not receive full credit for that task.

